

From a business idea to a buildable system

Week 2 • Requirements, Scope & GitHub

Define what the system should do — then create a disciplined team workspace to build it.



Where Week 2 fits with the textbook

This week is mostly instructor material. The textbook becomes central when JavaScript coding begins.

THIS WEEK

Required source:

- Instructor slides + guide
- In-class requirements exercises
- Git/GitHub guided practice

No JavaScript chapter is assigned yet.

TEXTBOOK CONNECTION

JavaScript for Web Warriors, 7e

Chapter 1 introduces JavaScript programming. We will begin that sequence when the course reaches coding.

For Week 2, the book is orientation only.

WHY THIS ORDER?

Before code, teams must agree on:

- the business problem
- users and stakeholders
- requirements
- scope
- business rules
- a collaboration workflow

Course foundation → textbook when it adds technical depth → capstone application every week.

By the end of this week, you should be able to...

Week 2 turns a promising project idea into a clear, testable, team-ready plan.

1 Define the business problem

Explain the need before proposing a solution.

2 Identify stakeholders

Recognize who uses, approves, supports, or is affected.

3 Write useful requirements

Separate functional and nonfunctional needs.

4 Write user stories + criteria

Turn vague needs into testable behavior.

5 Control project scope

Define what is inside — and outside — the semester project.

6 Use Git/GitHub basics

Create a repository, commit changes, sync, and collaborate.

Session 1 = define the system. Session 2 = organize the team and repository.

“Build me an app that makes returns easy.”

That sounds like a request — but is it enough to start building?

Who?

Who uses it — customer, employee, manager?

What transaction?

Return request, refund, exchange, or all three?

What rules?

Return window? Receipt? Product condition?

What outcome?

Submitted, approved, rejected, refunded?

A vague idea becomes buildable only after we make assumptions explicit.

Define the business problem before the technology

A solution is easier to evaluate when the problem is clear.

WEAK START

“We want a website with forms, dashboards, and notifications.”

This describes features, not the business problem.

It can lead to building impressive features that do not solve anything.

STRONG START

“Customers cannot see return eligibility or status, causing support calls and slow refunds.”

Now we can ask what information, rules, workflow, and controls are actually needed.

Problem statement = current pain + affected users + business impact.

Who has a stake in the transaction?

The “user” is important, but many other people and functions shape the requirements.

Initiator

Starts the transaction

Processor

Reviews or works the request

Approver

Authorizes an outcome

Owner

Owens the business process

Support

Maintains the system

Auditor / Risk

Needs evidence and control

Different stakeholders often want different things. Requirements reconcile those perspectives.

Functional vs. nonfunctional requirements

One describes behavior; the other describes qualities and constraints.

FUNCTIONAL — What must the system do?

Examples:

- submit a return request
- validate the purchase date
- calculate refund amount
- allow a manager to approve
- show transaction status

NONFUNCTIONAL — How well / under what constraints?

Examples:

- respond within 2 seconds
- work on mobile devices
- protect personal data
- keep an audit trail
- support 500 concurrent users

A strong specification needs both behavior and quality expectations.

A good requirement is clear, testable, and scoped

If nobody can tell whether it was satisfied, it is not a useful requirement.

Specific

Avoid “easy,” “fast,” or “user-friendly” without a definition.

Testable

You can observe a result and decide pass/fail.

Necessary

It solves the agreed problem or supports a required control.

Feasible

It fits the semester, skills, data, and technology available.

Unambiguous

Different readers should interpret it the same way.

Traceable

You can connect it to a stakeholder need or business rule.

Quick test: “How would we prove this requirement is met?”

Describe value from the user's perspective

User stories are compact conversation starters — not full specifications.

As a [role] I want [capability] so that [business value]

EXAMPLE

As an employee, I want to submit an expense request so that I can be reimbursed for approved business costs.

WHY IT HELPS

The story names:

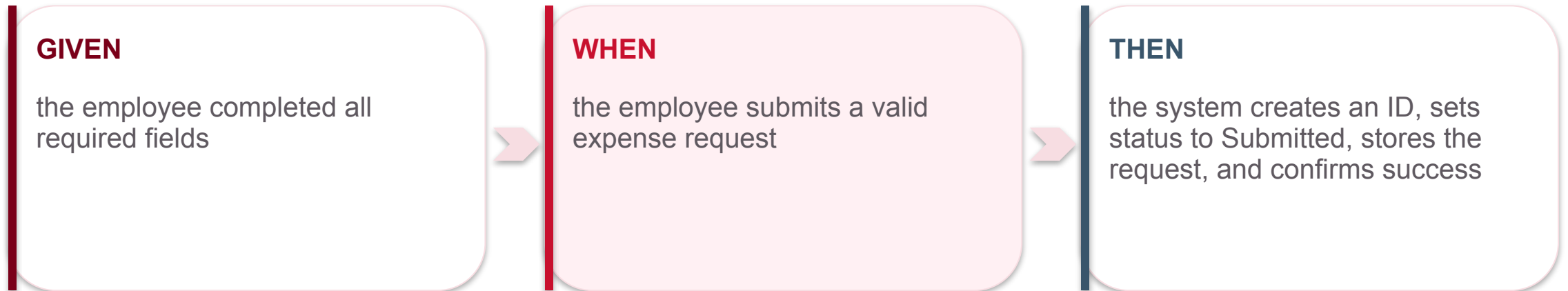
- who benefits
- what they need
- why it matters

It keeps the technology connected to business value.

A user story says “what and why.” Acceptance criteria define what “done” means.

Turn “done” into something testable

We can use a simple Given–When–Then structure to describe expected behavior.



WHY THIS MATTERS

The story tells us the goal. Acceptance criteria give developers and testers observable conditions for success.

If a criterion cannot be tested, rewrite it until it can.

Business rules become requirements — but they are not the same thing

Requirements say what the system must support; rules say what the organization allows or requires.

REQUIREMENT

“The system shall allow an employee to submit an expense request.”

This defines a capability the system must provide.

BUSINESS RULE

“A receipt is required for expenses above \$75.”

This defines a policy the transaction must obey.

In your capstone: each important business rule should map to one or more system behaviors or controls.

Define the boundary before the project grows

A semester project succeeds by doing one transaction well — not by imitating an entire enterprise.

MUST

Required for the core transaction

Create request
Validate rules
Track status

SHOULD

Valuable if time permits

Search history
Email-like notification

COULD

Optional enhancement

Charts
Extra filters
Theme options

WON'T — THIS TERM

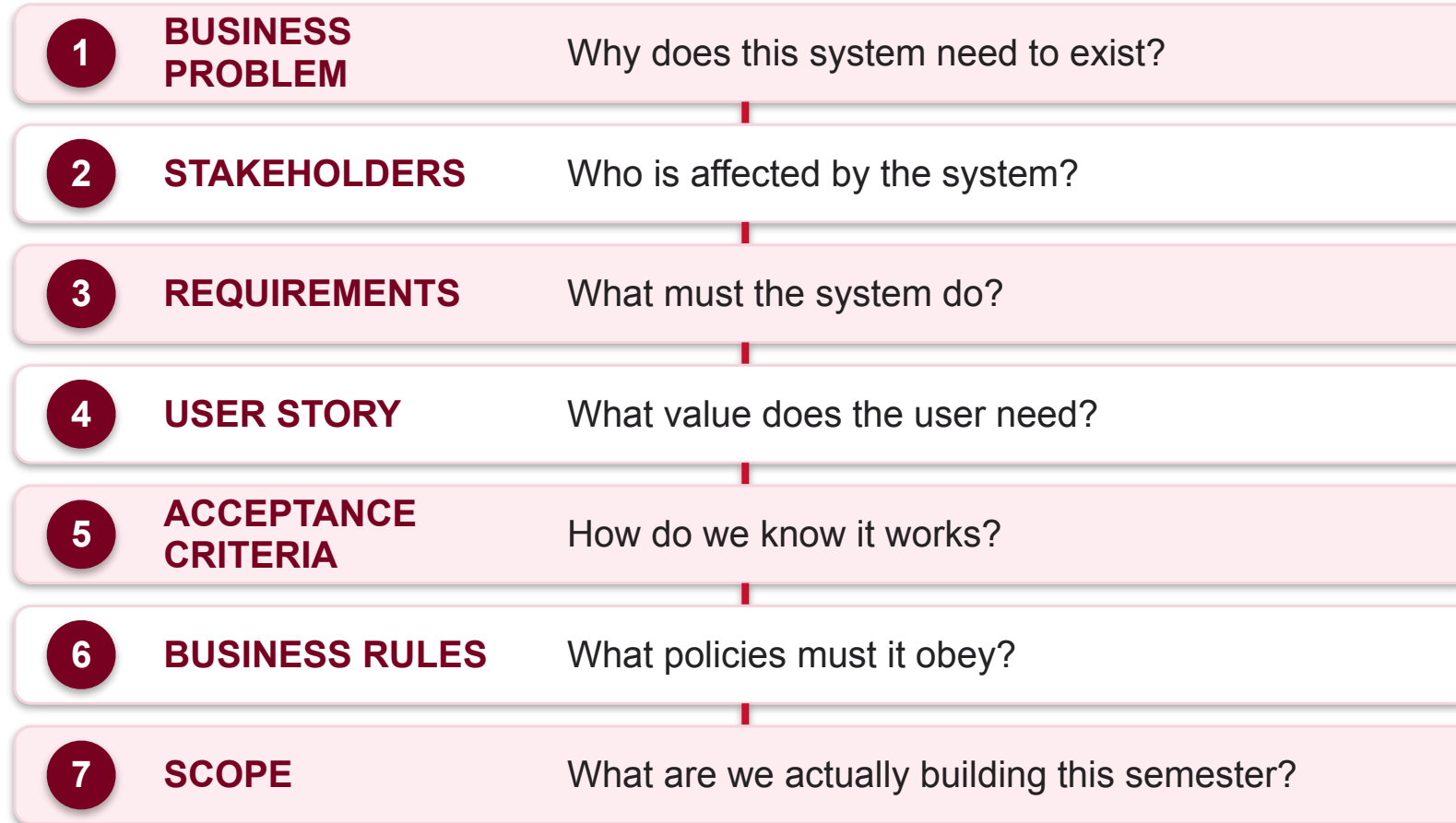
Explicitly out of scope

Real payments
Payroll integration
Production SSO

Writing “won’t” items is a professional design skill, not a sign of weakness.

Put the specification together

Before we move into design, connect the pieces into one coherent specification.



From vague idea to a usable specification

Expense reimbursement shows how all of the Week 2 pieces fit together.

Problem

Employees email receipts and cannot track status.

Stakeholders

Employee, manager, finance, auditor.

Requirements

System shall allow employees to submit expenses online and view current status.

User Story

As an employee, I want to submit expenses online so that I can track reimbursement.

Business Rules

Receipt required above \$75; director approval above \$5,000.

Acceptance Criteria

Valid submission receives a unique ID and Submitted status.

Scope

No real payroll/payment integration this semester.

Traceability

Every requirement, story, rule, and criterion should be justified by the problem and stakeholder needs.

Problem → stakeholders → requirements → user story → rules/criteria → scope.

Requirements Clinic

Turn a vague request into a small, testable specification.

1. Write the business problem
2. Identify 3 stakeholders
3. Write 2 functional + 1 nonfunctional requirement
4. Write 1 user story
5. Add 2 acceptance criteria
6. Define one “won’t” item

Before you build, can you answer these five questions?

Use this checklist on every capstone scenario.

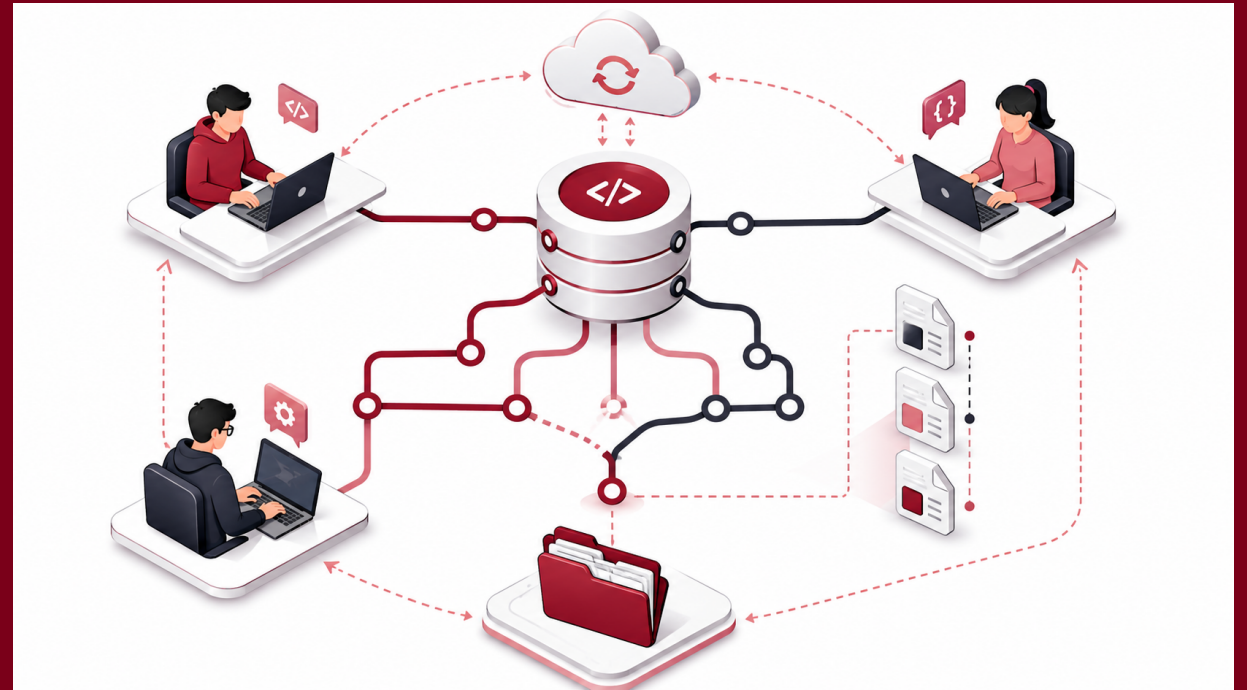
- 1 What business problem are we solving?
- 2 Who are the stakeholders?
- 3 What must the system do?
- 4 What rules and quality constraints apply?
- 5 What is explicitly out of scope?

Session 2: now we create the team workflow and repository that will carry this project all semester.

SESSION 2

From specification to team workspace

How version control turns individual changes into a reliable shared project.



What happens when four people edit the same project?

Without a disciplined history, teamwork quickly becomes file chaos.

WITHOUT VERSION CONTROL

final.html
final_v2.html
final_REAL.html
final_REAL_fixed.html

Nobody knows which copy is current, who changed what, or how to recover a working version.

WITH VERSION CONTROL

One repository
Visible history
Named commits
Separate branches
Controlled merges

The team can coordinate changes and recover earlier states.

Version control is not only a coding tool — it is a coordination and accountability system.

Git ≠ GitHub

They work together, but they solve different parts of the collaboration problem.

GIT

A distributed version-control system.

Git tracks the history of files and changes in a repository.

You can commit, branch, merge, inspect history, and work locally.

GITHUB

A hosted collaboration platform built around Git repositories.

GitHub adds remote hosting, permissions, issues, pull requests, reviews, and team collaboration.

Git tracks versions. GitHub helps teams share, review, and coordinate those versions.

Repository, commit, branch, remote

These four ideas are enough to understand most of the workflow we need this semester.

REPOSITORY

The project files plus their version history.

COMMIT

A named snapshot of a set of changes.

BRANCH

A separate line of development.

REMOTE

A shared repository location, such as GitHub.

The key mental model: local work creates history; the remote lets the team share it.

A commit is a meaningful checkpoint — not “Save”

Good commits make the history understandable and help isolate mistakes.



GOOD MESSAGE

“Add business rules to project README” > “changes” or “stuff fixed”

Commit when a coherent piece of work is complete and explain what changed.

Pull → change → stage → commit → push

For Week 2, understand the sequence before memorizing command syntax.



Later we will use commands; today the goal is to understand what each step accomplishes.

Branches let you isolate work before combining it

A branch protects the shared main line while a feature or fix is being developed.

main



WHY USE ONE?

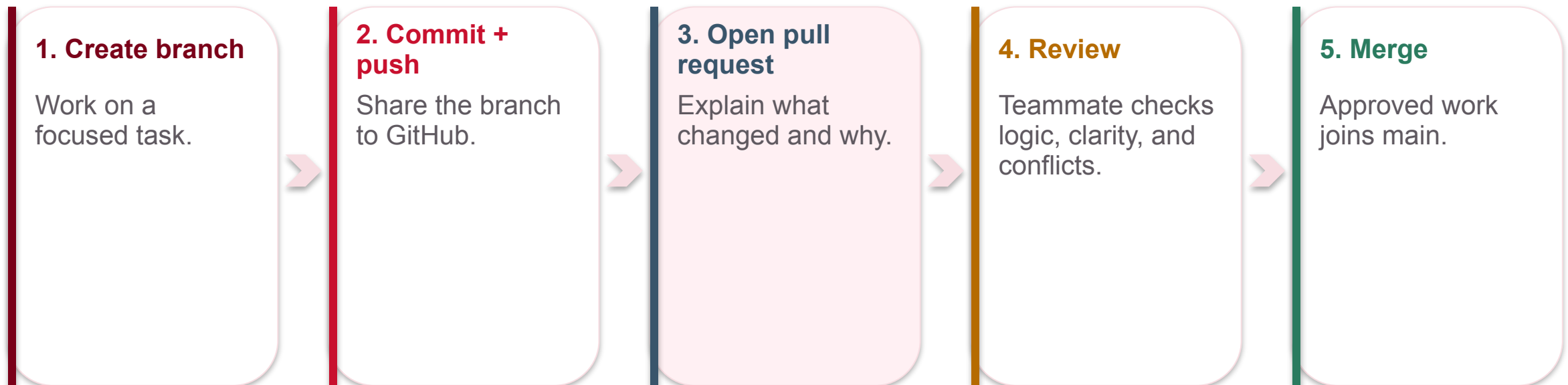
- work without breaking main
- review changes before merge
- reduce collisions
- create a clear ownership trail

For the capstone, branches should represent a feature or task — not a person forever.

Branch = safe workspace. Merge = intentionally combine approved changes.

A pull request is a review conversation before merge

Teams can propose, discuss, check, and then integrate a branch into the shared codebase.



Pull requests make teamwork visible — useful for learning, review, and individual contribution evidence.

Recommended team workflow for MIS 3371

Keep it simple enough to use every week.

Before work

Pull latest main

Create task branch

Example: feature/requirements

Work + commit

Small meaningful checkpoints

Push + PR

Explain change

Peer review

Another member checks it

Merge to main

Keep main stable

Rule: everyone contributes commits, but everyone must also understand the whole system.

Create the Week 2 team repository

Students follow along; the goal is a clean shared starting point.

- 1 Create a repository
- 2 Add a README
- 3 Add collaborators
- 4 Create a small branch
- 5 Edit README
- 6 Commit the change
- 7 Open a pull request
- 8 Merge and inspect history



Do not rush command syntax. Narrate what changed in the repository at each step.

Agree on how the team will work before conflict happens

A short team charter turns expectations into explicit working agreements.

Communication

Where do we communicate?
Expected response time?

Meetings

When and how often do we coordinate?

Work ownership

How are tasks assigned and rotated?

Git practice

Branch names, commit expectations, review rule.

Quality

Who checks requirements, testing, and documentation?

Conflict

What happens if work is late or uneven?

Roles can have primary owners, but no student is exempt from understanding the full application.

Start the repository clean

A predictable structure makes the semester easier to navigate and grade.

```
project-root/  
  README.md  
  docs/  
    proposal.md  
    requirements.md  
  diagrams/  
  src/ (later)  
  tests/ (later)
```

README SHOULD ANSWER

- What problem are we solving?
- What is the main transaction?
- Who is on the team?
- What is in scope?
- Where are the project documents?
- How do we run/use the application later?

The README evolves throughout the semester.

Good organization is part of professional practice — not cosmetic housekeeping.

Project Proposal + Team Charter + Repository

By the end of Week 2, each team should have one approved transaction and a shared workspace.

1 Business problem

Current pain + impact

2 Stakeholders

Who initiates, processes, approves, owns

3 Main transaction

One clear recordable event

4 Requirements

5 functional + 2 nonfunctional minimum

5 Rules + states

At least 5 rules; draft/ submitted/etc.

6 Scope

Must / should / won't

7 Team charter

Communication + responsibilities

8 GitHub repository

README + first reviewed change

The milestone is a design commitment — not a finished application.

What a strong Week 2 proposal looks like

Enough detail to guide the next several weeks without pretending every decision is final.

EXPENSE REIMBURSEMENT

Problem: email-based requests are slow and hard to track.

Transaction: submit an expense reimbursement request.

Users: employee, manager, finance.

Value: clearer status + fewer incomplete requests.

CORE REQUIREMENTS

- submit request
- attach / indicate receipt
- validate amount/date
- route based on threshold
- show current status

Nonfunctional:

- mobile-friendly
- auditable

SCOPE + RULES

Rules:

- amount > 0
- receipt > \$75
- director approval > \$5,000

Out of scope:

- real reimbursement payment
- HR/payroll integration

Good proposal = specific enough to build, small enough to finish, flexible enough to refine.

SESSION 2 ACTIVITY

Scope + Repository Checkpoint

Use the remaining class time to make your team ready for the milestone.

Team task — 12 minutes

1. Choose your best transaction
2. Write one-sentence problem statement
3. Add 3 stakeholders
4. Create / verify team repository
5. Put problem + team in README
6. Make one commit or PR as a team

Complete the first capstone milestone

This week's work should leave your team ready to design the transaction in more detail next week.

A CAPSTONE MILESTONE 1

Project proposal + team charter + repository

Submit:

- proposal document
- GitHub repository URL
- team charter

Main graded work this week.

B GIT/GITHUB CHECK

Each student should:

- accept collaborator access
- inspect history
- make or review at least one meaningful change
- know where project docs live

C CONCEPT CHECK

Short Canvas quiz:

- stakeholder vs user
- functional vs nonfunctional
- user story / criterion
- scope
- Git vs GitHub
- commit / branch / pull request

Textbook: no new required JavaScript chapter this week. Canvas will clearly mark when Chapter 1 begins.

Three ideas to carry into Week 3

You should now be able to explain both what your project is supposed to do and how your team will manage the work.

- 1 A requirement should be...** clear, testable, necessary, feasible, and scoped.
- 2 Git and GitHub are different because.** Git tracks versions; GitHub hosts and coordinates shared Git work.
- 3 Our capstone is now ready for...** deeper transaction modeling, data design, states, and architecture.

NEXT WEEK

Turn the approved proposal into a detailed transaction model, data dictionary, states, and architecture.